

Benchmarking gelu-tritonC, + cuda-geluC, + manual-geluC,
 + pytorch-gelu,
 profiling: gelu-tritonC, + gelu-cudaC, + manual-geluC,
 + pytorch-geluC,

⇒ Triton-geluC: block-size: # of block
 element-size: length of array

personal #5 approach to write kernel:

write kernel in python and compile it to Triton

compiled-geluC = torch.compile(~~manual~~ manual-geluC)

Benchmarking running time: for various gelu-xxx

element-wise operation

manual-time: 8.099 ms (not fuse python programs)
 pytorch-time: 1.109 ms
 cuda-time: 1.816 ms
 triton-time: 1.727 ms
 compiled-time: 1.49 ms

⇒ profiling (gelu-xxx)
 profiling (gelu-xxx)

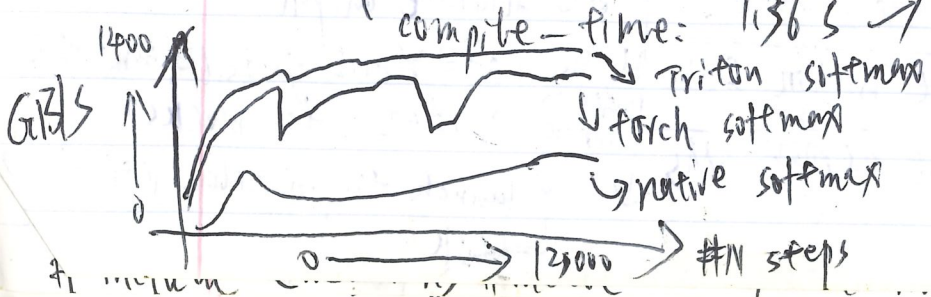
More advance computations: triton_softmax_mainC

row-wise operation

benchmarking

manual-time: 3.71 ms
 pytorch-time: 0.154 s
 triton-time: 1.92 s
 compile-time: 1.36 s

condim = 16384 operation
 = compiled-softmax



Summary for lecture #6:

- ①: Benchmarking to understand scaling:
 - # of steps
 - # of dim
 - # of batch
 - # of layer.
- ②: profiling for understanding internals of PyTorch functions
- ③: looking at PTX assembly to understand (bottoms out with kernels)
- ④: 5 ways to write a kernel function: (internals of CUDA kernels)
manual, PyTorch, CUDA, Triton, compiled python to Triton

cases:

⇒ Gelu: element-wise operation

⇒ softmax: row-wise operation

⇒ matmul: complex-aggregation

⇒ Key principle: organize computation to minimize reads/writes

⇒ Key ideas: kernel fusion (warehouse/factory analogy),
tiling & shared memory

⇒ Automatic compilers (Triton, torch.compile) will get better over time